

---

# APEX: Approximate-but-Exhaustive Search for Ultra-Large Combinatorial Synthesis Libraries

---

Aryan Pedawi<sup>\*1</sup> Jordi Silvestre-Ryan<sup>\*1</sup> Bradley Worley<sup>1</sup> Darren J. Hsu<sup>2</sup> Kushal S. Shah<sup>2</sup> Elias Stehle<sup>2</sup>  
Jingrong Zhang<sup>2</sup> Izhar Wallach<sup>1</sup>

## Abstract

Make-on-demand combinatorial synthesis libraries (CSLs) like Enamine REAL have significantly enabled drug discovery efforts. However, their large size presents a challenge for virtual screening, where the goal is to identify the top compounds in a library according to a computational objective (e.g., optimizing docking score) subject to computational constraints under a limited computational budget. For current library sizes—numbering in the tens of billions of compounds—and scoring functions of interest, a routine virtual screening campaign may be limited to scoring fewer than 0.1% of the available compounds, leaving potentially many high scoring compounds undiscovered. Furthermore, as constraints (and sometimes objectives) change during the course of a virtual screening campaign, existing virtual screening algorithms typically offer little room for amortization. We propose the approximate-but-exhaustive search protocol for CSLs, or APEX. APEX utilizes a neural network surrogate that exploits the structure of CSLs in the prediction of objectives and constraints to make full enumeration on a consumer GPU possible in under a minute, allowing for exact retrieval of approximate top- $k$  sets. To demonstrate APEX’s capabilities, we develop a benchmark CSL comprised of more than 10 million compounds, all of which have been annotated with their docking scores on five medically relevant targets along with physicochemical properties measured with RDKit such that, for any objective and set of constraints, the ground truth top- $k$  compounds can be identified and compared against the retrievals

from any virtual screening algorithm. We show APEX’s consistently strong performance both in retrieval accuracy and runtime compared to alternative methods.<sup>1</sup>

## 1. Introduction

The search for novel therapeutic agents is a cornerstone of modern medicine and drug discovery. In recent years, the emergence of ultra-large combinatorial synthesis libraries (CSLs), such as the Enamine REAL library, has significantly transformed this pursuit. These libraries, containing billions or even trillions of make-on-demand compounds, offer an unprecedented opportunity to explore a vast and diverse chemical space, significantly increasing the potential for identifying novel hit compounds with desirable properties. However, the sheer scale of these libraries presents a formidable challenge to traditional approaches to virtual screening.

State-of-the-art scoring functions used in virtual screening, like docking/affinity/co-folding scores, are too computationally expensive to render an exhaustive evaluation over modern CSLs, which number in the billions, practical. A number of virtual screening approaches have been developed to identify high-scoring compounds from large compound libraries under a limited evaluation budget. These methods include heuristic algorithms (Sadybekov et al., 2022), reinforcement learning (Pedawi et al., 2023; Klarich et al., 2024; de Oliveira et al., 2024), active learning (Graff et al., 2021; Mehta et al., 2021), and approaches that utilize generative models constrained to the library (Pedawi et al., 2022; Cretu et al., 2024; Luo et al., 2024; Gao et al., 2025). However, since these algorithms effectively assess only a small fraction of the total library—usually less than 1% of available compounds—they leave the vast majority of the chemical space unexplored and potentially overlook valuable compounds. Many of the listed strategies above include a surrogate modeling component, in which a more inference

---

<sup>\*</sup>Equal contribution <sup>1</sup>Work performed at Numerion Labs <sup>2</sup>NVIDIA. Correspondence to: Aryan Pedawi <pedawia@gmail.com>, Jordi Silvestre-Ryan <jordisr@berkeley.edu>.

---

<sup>1</sup>Code is available at <https://github.com/NumerionLabs/apex>. Please refer to the README for links to data and model weights.

efficient model (such as a neural network) is trained to approximate the oracle scoring function to enable exhaustive evaluation with the surrogate (Gentile et al., 2020; Graff et al., 2022). But this too is impeded by the size of modern CSLs, which would naively require billions of neural network evaluations to score exhaustively with the trained surrogate. Indeed, the growing size of CSLs and the computational demands of modern scoring functions in virtual screening create a pressing need for more efficient and comprehensive approaches.

At its core, virtual screening can be framed as a search problem, where the objective is to identify the top  $k$  compounds that optimize a specific scoring function while satisfying a set of program-specific constraints, namely desired physicochemical or ADMET properties like molecular weight, lipophilicity, and permeability. The ability to effectively handle constraints is particularly crucial in a virtual screening: for any given drug discovery project, the number of compounds in a screening library that violate these constraints can be orders of magnitude larger than those that satisfy them. This often complicates the workflow and can lead to the exploration of irrelevant chemical space or aggressive post-filtering.

In this work, we introduce APEX (approximate-but-exhaustive search), a new paradigm for searching ultra-large CSLs that enables fast, declarative queries. Once trained, an APEX model allows for efficient retrieval of the (approximate) top- $k$  compounds from a CSL according to a user-specified objective subject to a set of user-specified constraints. More than a virtual screening algorithm, APEX allows for low latency exploration of massive CSLs without the need for a complex, iterative workflow. The core of this capability is a neural network surrogate model that exploits the library’s combinatorial structure and amortizes the computation required for repeated querying, enabling real-time search across the entire enumerated CSL with remarkable efficiency on a modern GPU.

This paper details the theoretical underpinnings of the APEX methodology and demonstrates its practical application in virtual screening.

**Conflict of Interest Disclosure.** At the time this work was conducted, the authors AP, JSR, BR, and IW were employed by Numerion Labs, which develops and utilizes APEX as part of its virtual screening platform. The authors DH, KS, ES, and JZ are employed by NVIDIA, which develops CUDA, in which the factorized top- $k$  is implemented, as well as the NVIDIA GPUs that were utilized in experiments.

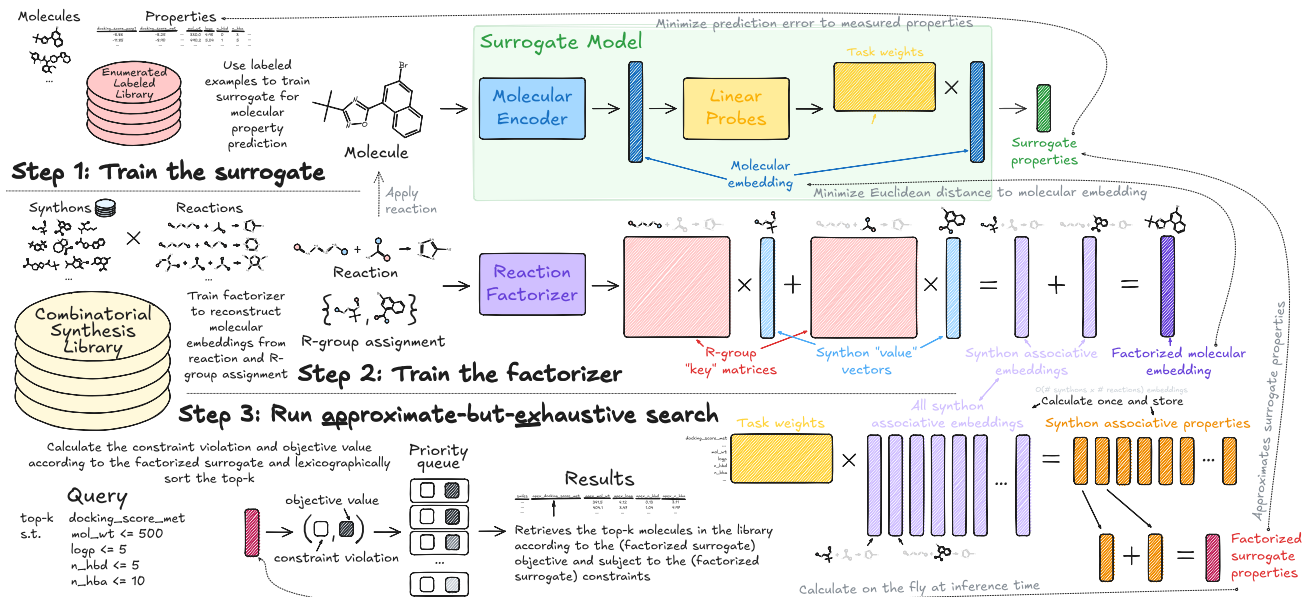
## 2. Related Work

**Virtual screening with surrogate models.** The use of neural network surrogates to approximate expensive scoring

functions has been widely explored as a strategy for accelerating virtual screening over large compound libraries. Deep Docking (Gentile et al., 2020) trains iterative QSAR models on a docked subset to progressively filter a library, reducing docking costs to a small fraction of the total. MolPal (Graff et al., 2021) frames virtual screening as a Bayesian optimization problem, using a surrogate trained on a growing labeled subset to guide acquisition over a fixed molecular pool; Graff et al. (2022) later extended this with active design-space pruning. MEMES (Mehta et al., 2021) combines Bayesian optimization with deep continuous molecular descriptors to achieve similar gains. These methods are designed for libraries of up to  $\sim 10^8$  molecules and scale linearly with library size during surrogate inference, making exhaustive evaluation with the surrogate a bottleneck as libraries grow into the billions. By exploiting the combinatorial structure of CSLs, APEX learns to decompose surrogate predictions into sums of pre-cached, per-synthon scalar contributions, reducing the cost of scoring any compound in the library to a small number of floating point operations, enabling exact retrieval of approximate top- $k$  sets via a single GPU pass over the full library in seconds.

**Protein-ligand representation learning for virtual screening.** A complementary line of work learns joint protein-ligand embeddings for virtual screening as a dense retrieval task. ConPLex (Singh et al., 2023) co-embeds proteins and drug molecules in a shared feature space using a pre-trained protein language model and contrastive learning against decoy compounds, enabling rapid shortlisting across large libraries. DrugCLIP (Gao et al., 2023; 2026) extends this idea by training on large-scale structural data to align pocket and ligand representations, enabling genome-scale screening at up to ten million times the speed of traditional docking. LigUnity (Feng et al., 2025) further unifies virtual screening and hit-to-lead optimization in a single foundation model through a hierarchical contrastive training objective. All of these methods require encoding library compounds individually; as Gao et al. (2023) notes, pre-encoding a 6-billion compound library takes on the order of 30 hours per target, with encoding time scaling linearly with library size. APEX avoids this bottleneck entirely by factorizing predictions over synthon contributions, and is complementary to these methods, as the factorization strategy could in principle be applied on top of such models to enable exhaustive screening against combinatorial synthesis libraries.

**Synthon-based and hierarchical screening approaches.** Several methods exploit the building-block structure of CSLs to reduce the effective search space. V-SYNTHES (Sadybekov et al., 2022) performs hierarchical structure-based screening by first docking minimal enumeration library fragments to identify promising scaffolds and then iteratively elaborating these with synthons to select high-scoring complete molecules, achieving greater than 1000-



**Figure 1.** The APEX (approximate-but-exhaustive) search protocol, enabling rapid, on-the-fly virtual screening of ultra-large CSLs. APEX consists of three main steps. **Step 1: Train the surrogate.** Given an enumerated and labeled dataset, a multi-task neural network is trained to predict molecular properties of interest, like docking scores. **Step 2: Train the factorizer.** Given a CSL, the reaction factorizer is trained to reconstruct embeddings of the surrogate model from reaction and R-group assignment pairs. The factorizer induces an approximation of surrogate properties that is amenable to substantial amortization in executing top- $k$  retrieval on ultra-large CSLs with respect to those properties. **Step 3: Run approximate-but-exhaustive search.** Given a search query (e.g., minimize docking score on target of interest subject to drug-likeness constraints), factorized surrogate properties are calculated for all compounds in the CSL and the top- $k$  are retrieved based on the objective subject to constraints. An efficient GPU implementation allows for running a top- $k$  search with  $k = 1$  million on a 10 billion compound CSL in approximately 30 seconds with a single T4 GPU.

fold acceleration over exhaustive docking on an 11-billion compound library. Thompson sampling (Klarich et al., 2024) treats each reaction’s synthon pool as a multi-armed bandit and maintains belief distributions over synthon score contributions to guide sequential evaluation. Unlike APEX, both approaches are iterative and require oracle evaluations during search; neither natively handles constraints on molecular properties. APEX, once trained, performs a single exhaustive pass over the full CSL using cached synthon embeddings, handles arbitrary combinations of objectives and constraints without additional oracle calls, and supports rapid amortization across queries.

**Generative models for synthesizable chemical space.** An orthogonal approach to large-scale drug discovery uses generative models constrained to synthesizable chemical space. Pedawi et al. (2022) introduced a graph generative model, CSLVAE, in which the decoder is constrained to only retrieve compounds from a CSL. NGT (de Oliveira et al., 2024) applies reinforcement learning over a CSLVAE latent space to execute a virtual screen over ultra-large CSLs. Other work, like SynFlowNet (Cretu et al., 2024), use GFlowNets to sample synthesis pathways from building blocks and reaction templates, and methods that project unconstrained generative outputs onto synthesizable analogs (Luo et al., 2024; Gao et al., 2025). It should be noted

that many of these approaches must effectively retrain or re-optimize the policy for each new screening campaign; changing a constraint or objective can often involve a new round of model fine-tuning or policy optimization, offering little amortization across screens. By contrast, APEX separates the one-time training cost from search execution, allowing modifications of query objectives and constraints to be run without any retraining and quickly.

**Factorization-based retrieval.** The use of additive or bilinear factorizations over structured input spaces has a well-established history in machine learning, most notably in recommender systems. Factorization machines (Rendle, 2010) and related matrix factorization methods (Koren et al., 2009) model feature interactions using factorized latent vectors, enabling efficient collaborative filtering at scale. APEX employs an analogous decomposition, i.e., the surrogate embedding of a product molecule is approximated as a sum of per-synthon associative contributions, reducing exhaustive evaluation over tens of billions of compounds to a cached lookup over a few hundred thousand synthon vectors. To our knowledge, APEX is the first method to apply factorization-based retrieval to virtual screening of combinatorial synthesis libraries.

### 3. Data

#### 3.1. Combinatorial Synthesis Libraries

A combinatorial synthesis library (CSL) is organized into a collection of multi-component *reactions*, each of which has a fixed number of components called *R-groups* which indicate placeholders for molecular building blocks called *synthons*. Hence, each product in a CSL can be identified by its reaction and R-group assignment. Due to their combinatorial design, commercially available make-on-demand CSLs such as the Enamine REAL library span a chemical space numbering in the tens of billions of compounds today from a few hundred thousand synthons. We refer the reader to Appendix A.1 for a self-contained description of the CSL formalism and the associated encoder hierarchy.

In this work, we designed our own open CSL as an alternative to existing proprietary CSLs for benchmarking and reproducibility purposes. We used a random sample of 1 million “lead-like” compounds from the ZINC22 database (Tingle et al., 2023) as a starting point for library construction. Our main focus here is on developing a large virtual library of valid CSL-like molecules, so we do not consider or ensure synthetic feasibility. We used the BRICS fragmentation algorithm (Degen et al., 2008), which breaks specific bonds based on defined chemical environments, to fragment each sampled molecule into two or three fragments. Each fragment is labeled with numbered pseudoatoms at the break points, with the BRICS rules determining which pseudoatom types can be joined to form a new bond. We applied the BRICS rules (as implemented in RDKit) to enumerate two- and three-component reactions that recombined these fragments into valid chemical products. This results in a set of fragmentation rules and fragments analogous to the reactions and synthons of a CSL.

Our final CSL comprises over 10B molecules and is by design evenly split between two- and three-component reactions. Additionally, we generated two smaller libraries by uniformly downsampling each reaction. These smaller libraries contain over 12M and 1M products and are fully enumerated to enable exhaustive docking and calculation of physicochemical properties. To address data leakage concerns, for all experiments in the paper, the surrogate is trained on the 1M compound CSL and evaluation is performed using either the 12M or 10B compound CSLs.

#### 3.2. Docking Scores and Physicochemical Properties

For benchmarking purposes, we selected five diverse protein targets: PARP1 (an enzyme), MET (a kinase), DRD2 (a GPCR), F10 (a protease), and ESR1 (a nuclear receptor). Receptor structures and binding sites were obtained from the DOCKSTRING dataset (García-Ortegón et al., 2022). Molecules from this smaller library were embedded with

RDKit and docked against these five targets using an accelerated implementation of the AutoDock Vina (Trott & Olson, 2010) scoring function designed to run on GPU (Morrison et al., 2020). In addition to docking scores, we calculated various physicochemical properties (e.g., molecular weight, number of hydrogen bond donors and acceptors; full list can be seen in Figure 6) for each molecule in this enumerated library.

### 4. Methodology

Given a CSL  $\mathcal{D}$ , which defines the chemical space  $X_{\mathcal{D}}$  of eligible compounds, our goal is to identify the top- $k$  compounds from the library that maximize an objective subject to constraints. This retrieval problem can be expressed as

$$\begin{aligned} X_k^* := \arg \max_{\substack{X_k \subset X_{\mathcal{D}} \\ |X_k| \leq k}} \sum_{\mathbf{x} \in X_k} f_0(\mathbf{x}), \\ \text{subject to } \ell_i \leq f_i(\mathbf{x}) \leq u_i, \\ \forall \mathbf{x} \in X_k, i = 1, \dots, m, \end{aligned} \quad (1)$$

where  $f_0 : X \rightarrow \mathbb{R}$  is the objective and  $f_i : X \rightarrow \mathbb{R}$ ,  $i = 1, \dots, m$ , are the constraints with bounds  $\ell_i < u_i$ . This problem is complicated by the present and rapidly growing size of CSLs,  $|X_{\mathcal{D}}| > 10^{10}$ , combined with the fact that many objectives and constraints of interest—such as docking or co-folding scores—are computationally expensive to evaluate, which precludes exhaustive evaluation. We can relax (1) by substituting the original objective and constraints with surrogate models:

$$\begin{aligned} \hat{X}_k^* := \arg \max_{\substack{X_k \subset X_{\mathcal{D}} \\ |X_k| \leq k}} \sum_{\mathbf{x} \in X_k} \hat{f}_0(\mathbf{x}), \\ \text{subject to } \ell_i \leq \hat{f}_i(\mathbf{x}) \leq u_i, \\ \forall \mathbf{x} \in X_k, i = 1, \dots, m, \end{aligned} \quad (2)$$

Neural network surrogates that operate directly on a 2D molecular graph or a 1D representation like SMILES are a good choice, but exhaustive evaluation of ultra-large CSLs with such surrogates is still far from a routine computational task, requiring  $O(|X_{\mathcal{D}}|)$  neural network evaluations.

We develop a surrogate-based modeling strategy that permits (2) to be solved efficiently for ultra-large CSLs. First, let us discuss the parameterization of the surrogate models admissible under this design.

#### 4.1. Surrogate Model Parameterization

Let  $g_\theta : X \rightarrow \mathbb{R}^d$  be a neural network that encodes a molecule  $\mathbf{x} \in X$  into a  $d$ -dimensional embedding space. We place no restrictions on  $g_\theta$  beyond this, i.e., it can be a transformer that operates on the SMILES representation of  $\mathbf{x}$ , a GNN that operates on a 2D graph representation of

$\mathbf{x}$ , or some other similarly appropriate choice. We model each task  $i = 0, \dots, m$  as a linear function of the molecular embedding,

$$\hat{f}_i(\mathbf{x}) = \mathbf{w}_i^\top g_\theta(\mathbf{x}) + b_i, \quad (3)$$

where  $\mathbf{w}_i \in \mathbb{R}^d$  and  $b_i \in \mathbb{R}$ . Given labeled data from each task, written  $p_i(\mathbf{x}, y)$  where  $y = f_i(\mathbf{x})$ , the surrogate model is trained to minimize the prediction error relative to ground truth:

$$\min_{\theta, \mathbf{W} \in \mathbb{R}^{(m+1) \times d}, \mathbf{b} \in \mathbb{R}^{m+1}} L(\theta, \mathbf{W}, \mathbf{b}), \quad (4)$$

where

$$L(\theta, \mathbf{W}, \mathbf{b}) = \sum_{i=0}^m \mathbb{E}_{p_i(\mathbf{x}, y)} \mathbb{E}_{p(\boldsymbol{\varepsilon})} \left[ (\mathbf{w}_i^\top (g_\theta(\mathbf{x}) + \boldsymbol{\varepsilon}) + b_i - y)^2 \right]. \quad (5)$$

The surrogate is trained with noise added to the embeddings, sampled from a simple distribution  $p(\boldsymbol{\varepsilon})$  like an isotropic normal. The relevance of this particular detail will be explained shortly.

## 4.2. Factorization of Surrogate Embeddings

As a review (see Pedawi et al. (2022) for additional details), we can represent a CSL  $\mathcal{D} \equiv (T, R, S, \psi, \phi)$  hierarchically, with *synthons* (molecular fragments)  $S$  at the bottom of the hierarchy, *R-groups*  $R$  in the middle, and *reactions*  $T$  at the top. Every synthon index  $s \in S$  is associated with a corresponding molecular representation  $\mathbf{x}_s \in X_*$  (again, SMILES or 2D graph), where  $X_* \supset X$  extends  $X$  to include attachment points, represented by the token “\*”. An R-group, denoted by the index  $r \in R$ , is comprised of a set of synthons that constitute valid assignments to the associated component in a multi-component reaction. A multi-component reaction  $t \in T$ , together with a valid assignment of synthons to their constituent R-groups, produces a single molecule via chemical synthesis as output. We denote by  $\psi_{T \rightarrow R} : T \rightarrow \mathcal{P}(R)$  the function that returns the set of R-groups  $\psi_{T \rightarrow R}(t) \subset R$  associated with a reaction  $t$ , where  $\mathcal{P}(\cdot)$  denotes the power set function. Similarly,  $\psi_{R \rightarrow S} : R \rightarrow \mathcal{P}(S)$  returns the set of synthons  $\psi_{R \rightarrow S}(r) \subset S$  that can be assigned to a particular R-group. Each molecule in  $\mathcal{D}$  can be referenced by a multi-index, denoted by  $\chi = (t, \{(r, s) : \exists s \in \psi_{R \rightarrow S}(r), \forall r \in \psi_{T \rightarrow R}(t)\})$ , which describes the reaction and R-group assignment used to construct the molecule,  $\mathbf{x} := \phi(\chi)$ .

We utilize the design proposed in Pedawi et al. (2022) to model an associated hierarchy of representations that describe the library at these three levels of resolution. First,

the `SynthonEncoder` :  $X_* \rightarrow \mathbb{R}^{d_S}$  produces an embedding for each synthon  $s$  as a function of its molecular representation  $\mathbf{x}_s$ . Next, a deep set network called the `RgroupEncoder` :  $\mathbb{R}^{d_S} \times \dots \times \mathbb{R}^{d_S} \rightarrow \mathbb{R}^{d_R}$  produces an embedding for each R-group  $r$  as a function of the representations of its constituent synthons. Finally, another deep set network, `ReactionEncoder` :  $\mathbb{R}^{d_R} \times \dots \times \mathbb{R}^{d_R} \rightarrow \mathbb{R}^{d_T}$ , produces an embedding for each reaction  $t$  as a function of the representations of its constituent R-groups. This is described by the following computational stack:

$$\mathbf{h}_s^S = \text{SynthonEncoder}(\mathbf{x}_s), \quad (6)$$

$$\mathbf{h}_r^R = \text{RgroupEncoder}(\{\mathbf{h}_s^S : \forall s \in \psi_{R \rightarrow S}(r)\}), \quad (7)$$

$$\mathbf{h}_t^T = \text{ReactionEncoder}(\{\mathbf{h}_r^R : \forall r \in \psi_{T \rightarrow R}(t)\}). \quad (8)$$

From these representations, we aim to reconstruct the molecular embedding  $g_\theta(\phi(\chi))$  as a function of the associated multi-index  $\chi$  in a manner which will permit fast and efficient approximations of (3). To do this, we model the embedding space of  $g_\theta$  via a linear associative map of the R-group assignments. In particular, we introduce a `SynthonValueEncoder` :  $\mathbb{R}^{d_S} \rightarrow \mathbb{R}^{d_U}$  and `RgroupKeyEncoder` :  $\mathbb{R}^{d_R} \times \mathbb{R}^{d_T} \rightarrow \mathbb{R}^{d \times d_U}$  which produce intermediate representations that are combined as follows to arrive at a prediction of the molecule’s latent representation:

$$\mathbf{v}_s = \text{SynthonValueEncoder}(\mathbf{h}_s^S), \quad (9)$$

$$\mathbf{K}_r = \text{RgroupKeyEncoder}(\mathbf{h}_r^R, \mathbf{h}_{\psi_{R \rightarrow T}(r)}^T), \quad (10)$$

$$\mathbf{u}_{r,s} = \mathbf{K}_r \mathbf{v}_s, \quad (11)$$

$$\hat{g}_\lambda(\chi) = \sum_{(r,s) \in \chi} \mathbf{u}_{r,s}. \quad (12)$$

The `SynthonEncoder`, `RgroupEncoder`, `ReactionEncoder`, `SynthonValueEncoder`, and `RgroupKeyEncoder` all combine to form the `ReactionFactorizer` or just the “factorizer” for short, which we represent by the function  $\hat{g}_\lambda(\chi)$ . Given a library  $\mathcal{D}$  and the frozen surrogate encoder  $g_\theta$ , we train the factorizer to minimize the reconstruction error of the surrogate embeddings,

$$\min_{\lambda} \mathbb{E}_{p(\chi|\mathcal{D})} [\|g_\theta(\phi(\chi)) - \hat{g}_\lambda(\chi)\|_2^2]. \quad (13)$$

### 4.3. Putting it Together

We can factorize the surrogate predictions by substituting (12) into (3), which simplifies as follows:

$$\hat{f}_i(\chi) = \mathbf{w}_i^\top \hat{g}_\lambda(\chi) + b_i, \quad (14)$$

$$= \mathbf{w}_i^\top \left( \sum_{(r,s) \in \chi} \mathbf{u}_{r,s} \right) + b_i, \quad (15)$$

$$= \sum_{(r,s) \in \chi} \mathbf{w}_i^\top \mathbf{u}_{r,s} + b_i, \quad (16)$$

$$= \sum_{(r,s) \in \chi} v_{i,r,s} + b_i, \quad (17)$$

where the  $v_{i,r,s}$  terms are called *synthon associative contributions*. We use the shorthand  $\hat{f}_i(\mathbf{x})$  to denote  $\hat{f}_i(\phi(\chi))$  when  $\mathbf{x} = \phi(\chi)$ , i.e., we can express  $\hat{f}_i : X_{\mathcal{D}} \rightarrow \mathbb{R}$ . We call the expression in (17) the *approximate-but-exhaustive (APEX) factorization*, because it permits us to solve the top- $k$  problem (2) under the approximation (17) via exhaustive evaluation on  $\mathcal{D}$ :

$$\begin{aligned} \hat{X}_k^* &:= \arg \max_{\substack{X_k \subset X_{\mathcal{D}} \\ |X_k| \leq k}} \sum_{\mathbf{x} \in X_k} \hat{f}_0(\mathbf{x}), \\ \text{subject to } &\ell_i \leq \hat{f}_i(\mathbf{x}) \leq u_i, \\ &\forall \mathbf{x} \in X_k, i = 1, \dots, m. \end{aligned} \quad (18)$$

Since the surrogate is trained with noise added to the embeddings as per (5) (and therefore learns embeddings whose linear projections are robust to such perturbations), the APEX prediction induced by the substitution in (14) is robust to the so-called errors-in-variables problem (Griliches, 1974). The addition of isotropic normal noise in (5) is therefore a technique to statistically regularize the surrogate to ensure that it remains robust to the subsequent factorization.

To demonstrate, let’s consider a simplified CSL comprised of a single three-component reaction with 10,000 distinct synthons for each R-group, i.e.,  $|S| = 30,000$ . This yields a total of one trillion products in  $\mathcal{D}$ . Exhaustive screening with  $\hat{f}_\theta$  would therefore require one trillion neural network evaluations. APEX, on the other hand, produces all intermediate representations for the library with just 30,000 neural network evaluations. The associative embeddings (11) can be cached as a  $|S| \times d$  matrix for later re-use. Supposing  $d = 1024$ , this would require about 120 MB of memory. In contrast, to cache the latent representations for all of the one trillion products in  $\mathcal{D}$  would require about 4 PB of memory. With the associative embeddings in our possession, we can calculate their dot products with the task weight  $\mathbf{w}_i$ , which is just  $2|S| \times d - |S| = 61.41$  million floating point operations. Once these terms have been computed, each  $\hat{f}_i(\mathbf{x})$  can be calculated with just a few floating point operations

(three in this case: the summation of the three synthon associative contributions and the bias term  $b_i$ ). Hence, we can approximate the surrogate predictions for all compounds in  $\mathcal{D}$  with just three trillion floating point operations (i.e., 3 TFLOP). Noting that the NVIDIA Tesla T4 GPU is able to perform 8.1 TFLOPS, the APEX factorization (17) theoretically permits evaluation of all one trillion compounds in the CSL in just a few seconds.

To construct the top- $k$  set  $\hat{X}_k^*$  for the retrieval problem (18), we can stream the pre-computed synthon associative contributions for each task and add them with the bias to form the APEX prediction (17). We can compute the constraint violation under the APEX predictions,

$$\begin{aligned} \hat{c}(\mathbf{x}) &= - \sum_{i=1}^m \max(0, \ell_i - \hat{f}_i(\mathbf{x})) \\ &\quad - \sum_{i=1}^m \max(0, \hat{f}_i(\mathbf{x}) - u_i), \end{aligned} \quad (19)$$

which is zero if all of the predicted constraints are satisfied and negative if there is any violation. Hence, for each compound in  $\mathcal{D}$ , we form a two-dimensional vector  $(\hat{c}(\mathbf{x}), \hat{f}_0(\mathbf{x}))$  that is used to enter compounds into a priority queue of size  $k$  that organizes them in lexicographical order. Once we have exhausted through all compounds in  $\mathcal{D}$ , we can remove any compound from the top- $k$  set where  $\hat{c}(\mathbf{x}) < 0$ . The result is the solution  $\hat{X}_k^*$  to (18).

### 4.4. Top- $k$ Retrieval

The exposition in the previous subsection on runtime only considers evaluation of APEX predictions over the entire CSL and ignores overhead introduced by maintenance of the top- $k$  set. APEX implements custom top- $k$  algorithms for the CPU and GPU within a PyTorch CUDA C++ extension module. The CPU PyTorch operator calculates each molecule’s APEX objective  $\hat{f}_0(\mathbf{x})$  and constraint violation  $\hat{c}(\mathbf{x})$  on the fly and streams them directly into a priority queue. However, APEX is uniquely suited for the GPU, as it requires only a small initial data transfer from CPU to GPU, and all intermediate calculations can be performed entirely on-GPU. To leverage the high compute capability and memory bandwidth of the GPU, the CUDA PyTorch operator employs a chain-of-batches strategy with the GPU-compatible AIR top- $k$  algorithm (Zhang et al., 2023). Additional details are provided in Appendix A.5.

## 5. Evaluation

To demonstrate APEX’s capabilities on a variety of pertinent virtual screening queries, we evaluate its ability to accurately retrieve the top- $k$  compounds in a large, representative CSL

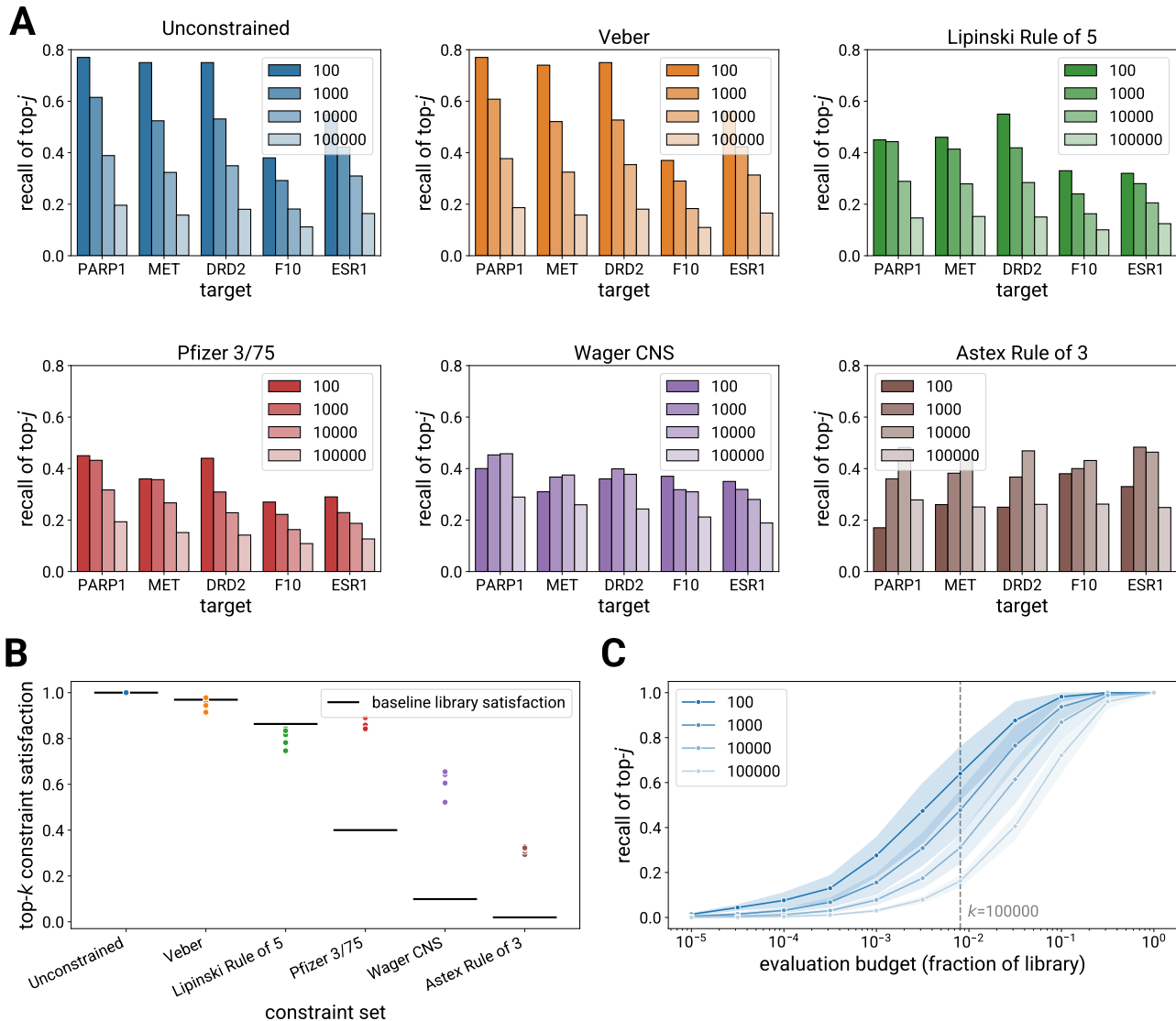


Figure 2. (A) Percent of compounds in the ground truth top- $j$  set retrieved by the APEX top  $k = 100,000$  set from the 12M compound CSL. A random baseline will achieve a recall below 0.01. (B) Constraint satisfaction rates for the APEX retrievals. Black line denotes the base fraction of satisfying compounds in the library for each set of constraints. (C) Recall of different top- $j$  sets without constraints as a function of increasing evaluation budget. Recall is averaged over all five targets, with error bars showing the standard deviation. Per-target recall curves are shown in Figure 10 of the Appendix. The dashed line corresponds to  $k = 100,000$ , the budget used for (A) and (B).

by docking score across the five selected targets (PARP1, MET, DRD2, F10, and ESR1) and against a number of relevant constraint sets used in drug discovery (described in Appendix A.7).

In all reported experiments, the surrogate is trained on the 1M compound CSL described in Section 3 (this is the only step in which labels are provided to the model) and the factorizer is trained on either the 12M or 10B compound CSL (in the absence of labels) to reconstruct embeddings produced by the trained surrogate model. We use an embedding dimension of  $d = 64$ . No extensive hyperparameter tuning was performed; we opted for a lightweight model for

purposes of demonstrating APEX (but note that runtime for APEX search is not a function of  $d$  once pre-calculations have been performed).

### 5.1. Top- $k$ Retrieval

For a library  $\mathcal{D}$ , objective  $f_0$ , constraints  $\{(f_i, \ell_i, u_i)\}_{i=1}^m$ , and evaluation budget  $k$ , we are ultimately interested in a screening algorithm’s ability to accurately retrieve the ground truth top- $j$  set  $X_j^*$  in (1), where  $j \leq k$ . For example, we might have the budget to evaluate  $k = 100,000$  compounds but wish to quantify what percent of the top- $j = 100$

were correctly retrieved. For APEX, this quantity can be expressed simply as

$$\text{Recall-}j\text{-at-}k = \frac{|X_j^* \cap \hat{X}_k^*|}{|\hat{X}_k^*|}. \quad (20)$$

Of course, this requires knowing the ground truth top- $j$  set  $X_j^*$  for a given search query. We use the 12M enumerated and exhaustively scored CSL to perform such an evaluation.

Results are shown in Figure 2A. With a budget of  $k = 100,000$  retrievals (representing 0.803% of compounds in the CSL), the ground truth top- $j$  compounds are recovered at rates far exceeding selection with a random baseline across all targets and for all sets of constraints. In addition to the search without constraints, the recall is highest for the Veber set of constraints, which are the least stringent and are satisfied by most compounds in the library (Figure 2B). The Astex Rule-of-3 constraints are designed for fragment-based drug discovery but we include them here as an example of a more stringent constraint set. While the rate of constraint satisfaction is lowest for this set, it is still much higher than the baseline rate of constraint satisfaction in the library.

## 5.2. Comparison with Thompson Sampling

We compare APEX against the Thompson sampling (TS) algorithm of Klarich et al. (2024), which was specifically designed for screening ultra-large CSLs. Thompson sampling (Thompson, 1933) is a classic Bayesian decision-making strategy in which a belief distribution over expected rewards is maintained for each action, an action is selected by sampling from these beliefs, and the distribution is updated upon observing the outcome. In the context of CSL screening, Klarich et al. (2024) instantiate this principle in synthon space: for each reaction, independent belief distributions are maintained over the expected reward contribution of each synthon, synthon combinations are sampled according to these beliefs and evaluated with the oracle, and the distributions are updated accordingly.

Crucially, this TS implementation operates separately on each reaction, i.e., the multi-armed bandit is defined at the level of a single reaction’s synthon pool, and results are aggregated across reactions post hoc, which limits its ability to jointly optimize over the full library. As TS is run on each reaction separately, we limit the comparison to the top five largest reactions in our 12M CSL (in total comprising over 4 million products). The total number of evaluations for TS is  $|S| \times w + i$ , where  $S$  is the set of synthons for that particular reaction,  $w$  is the number of warmup steps, and  $i$  is the number of TS iterations and output molecules.

We run TS for 100, 1,000, or 10,000 iterations, with 3 warmup steps for two-component reactions and 10 for three-component reactions (as suggested in Klarich et al. (2024)).

As this TS implementation does not directly support constraints on molecular properties, we perform the comparison in the unconstrained case for both APEX and TS, only minimizing docking score as the objective. Full results are shown in Figure 9 in the Appendix. While results vary across targets and reactions, APEX consistently outperforms or matches TS at recalling the top- $j$  compounds, showing particular strength at lower evaluation budgets.

## 5.3. Docking Score Enrichment on Ultra-Large Libraries

Figure 7 plots the empirical CDF of docking scores across the five targets for the APEX top- $k$  set in both the 10B and 12M compound CSLs against the background distribution of scores from the 12M compound library. This result demonstrates clear enrichment in the APEX top- $k$  sets relative to the background set, and further highlights the value of screening larger CSLs to identify higher scoring compounds enabled by APEX’s accelerated runtime and ability to scale to ultra-large combinatorial libraries.

### 5.3.1. ZERO-SHOT APPLICATION TO THE ENAMINE REAL LIBRARY

In addition to the BRICS CSLs, we also apply APEX to the commercial Enamine REAL library (9-2024 version). This library contains more than 70B compounds and serves as a test of APEX’s generality, both in scaling to even larger library sizes and as an application of a pretrained surrogate and factorizer in a zero-shot manner.

Figure 7 presents the docking score distributions from this library alongside a background score distribution generated from 100,000 random compounds. Despite the surrogate and factorizer being trained on a different, much smaller library, APEX is able to enrich docking scores with respect to the background distribution of the Enamine library and, in most cases, with respect to the top- $k$  of the 10B BRICS library. The lowest enrichment is from MET kinase, which also corresponds to the largest drop-off in R-squared in this zero-shot application of the factorizer (Figure 6 in the Appendix). While these results demonstrate APEX’s capabilities in a zero-shot context on ultra-large vendor CSLs today, even greater docking score enrichment is likely achievable through fine-tuning the surrogate (and subsequently the factorizer) using labeled data from the target CSL.

## 5.4. Execution Speed of APEX on Ultra-Large CSLs

Table 1 reports runtimes of APEX top- $k$  search on the BRICS and Enamine libraries for different choices of  $k$ , evaluated on a single NVIDIA Tesla T4 GPU. The reported runtimes represent end-to-end execution, i.e., from problem specification to an output dataframe with APEX top- $k$  SMILES and their associated APEX-predicted objective



and constraint values. In screening the 10B and 70B compound libraries, we observe an order of magnitude speedup in runtime when using the GPU top- $k$  implementation as opposed to CPU. Further, as constraints are included, the gap widens significantly, with the CPU implementation’s runtime increasing approximately linearly in the number of constraints added. Using the GPU top- $k$  implementation, APEX is able to retrieve the approximate top  $k = 1,000,000$  compounds from a 10B compound library in less than thirty seconds under standard drug likeness constraints, making it a highly performant and scalable search protocol for ultra-large CSLs.

| Unconstrained      |             |              |                |
|--------------------|-------------|--------------|----------------|
| $k$                | BRICS 12M   | BRICS 10B    | REAL 70B       |
| 10,000             | 0.3 (0.4)   | 10.9 (130.7) | 168.4 (838.5)  |
| 100,000            | 1.2 (1.2)   | 11.6 (131.4) | 169.3 (847.2)  |
| 1,000,000          | 10.9 (12.7) | 21.2 (147.6) | 184.0 (858.9)  |
| Lipinski Rule of 5 |             |              |                |
| $k$                | BRICS 12M   | BRICS 10B    | REAL 70B       |
| 10,000             | 0.3 (0.7)   | 13.9 (437.7) | 186.1 (3163.2) |
| 100,000            | 0.9 (1.7)   | 14.6 (443.7) | 187.5 (3184.6) |
| 1,000,000          | 10.8 (12.9) | 24.3 (462.2) | 202.4 (3142.5) |

Table 1. Runtime of APEX top- $k$  search across constraints and library sizes in seconds. Times are averaged over five runs (one with each target’s docking score as an objective), with GPU runtime reported first and CPU runtime reported in parentheses.

## 6. Conclusion

In this paper, we proposed the APEX search protocol for the virtual screening of combinatorial synthesis libraries, enabling the rapid execution of declarative queries that scales to ultra-large libraries (in excess of 10 billion compounds). While traditional virtual screening algorithms are limited by design to evaluate only a small fraction of the eligible search space, APEX enables a fast, exhaustive evaluation over the entire search space by taking advantage of the structure of CSLs. This allows researchers to rapidly identify high-scoring compounds virtually that satisfy design constraints. We demonstrated APEX’s capabilities on a benchmark CSL of over 10 million compounds, all annotated with ground truth docking scores and physicochemical properties. Our results show that APEX consistently achieves high recall rates for the ground truth top- $k$  compounds at low  $k$  and effectively satisfies diverse constraint sets, far exceeding random baselines.

APEX is a significant step towards making exhaustive virtual screening a routine computational task. Its ability to efficiently screen entire CSLs ensures that valuable, high-scoring compounds are not overlooked. Moreover, due to its rapid execution speed—virtually screening a CSL in ex-

cess of 10 billion compounds in less than 30 seconds with a single Tesla T4 GPU—APEX enables rapid hypothesis testing and interactive exploration of chemical space.

## Acknowledgments

The authors thank David Graff and Henry van den Bedem for early conceptual input, Yurii Moroz for feedback on library construction, and Kate Stafford for discussions on the manuscript, experiments, and applications.

## Impact Statement

This paper introduces the APEX search protocol to enable the rapid, exhaustive evaluation of ultra-large combinatorial synthesis libraries (CSLs) containing tens of billions of compounds. This technology lowers the computational barriers to exploring such massive, make-on-demand chemical spaces, potentially accelerating the identification of novel therapeutic agents.

## References

- Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S., and Hopkins, A. L. Quantifying the chemical beauty of drugs. *Nature Chemistry*, 4(2):90–98, 2012.
- CCCL Development Team. *CCCL: CUDA C++ Core Libraries*, 2023. URL <https://github.com/NVIDIA/cccl>.
- Congreve, M., Carr, R., Murray, C., and Jhoti, H. A “rule of three” for fragment-based lead discovery? *Drug Discovery Today*, 8(19):876–877, 2003.
- Cretu, M., Harris, C., Roy, J., Bengio, E., and Liò, P. Synflownet: Towards molecule design with guaranteed synthesis pathways. In *ICLR 2024 Workshop on Generative and Experimental Perspectives for Biomolecular Design*, 2024.
- de Oliveira, S. H., Pedawi, A., Kenyon, V., and van den Bedem, H. NGT: Generative AI with synthesizability guarantees discovers MC2R inhibitors from a tera-scale virtual screen. *Journal of Medicinal Chemistry*, 67(21):19417–19427, 2024.
- Degen, J., Wegscheid-Gerlach, C., Zaliani, A., and Rarey, M. On the art of compiling and using “drug-like” chemical fragment spaces. *ChemMedChem*, 3(10):1503, 2008.
- Ertl, P. and Schuffenhauer, A. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of Cheminformatics*, 1(1):8, 2009.

- Feng, B., Liu, Z., Li, H., Yang, M., Zou, J., Cao, H., Li, Y., Zhang, L., and Wang, S. Hierarchical affinity landscape navigation through learning a shared pocket-ligand space. *Patterns*, 6:101371, 2025. doi: 10.1016/j.patter.2025.101371.
- Gao, B., Qiang, B., Tan, H., Jia, Y., Ren, M., Lu, M., Liu, J., Ma, W.-Y., and Lan, Y. DrugCLIP: Contrastive protein-molecule representation learning for virtual screening. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Gao, B. et al. Deep contrastive learning enables genome-wide virtual screening. *Science*, 2026. doi: 10.1126/science.ads9530.
- Gao, W., Luo, S., and Coley, C. W. Generative ai for navigating synthesizable chemical space. *Proceedings of the National Academy of Sciences*, 122(41):e2415665122, 2025.
- García-Ortegón, M., Simm, G. N., Tripp, A. J., Hernández-Lobato, J. M., Bender, A., and Bacallado, S. DOCK-STRING: Easy molecular docking yields better benchmarks for ligand design. *Journal of Chemical Information and Modeling*, 62(15):3486–3502, 2022.
- Gentile, F., Agrawal, V., Hsing, M., Ton, A.-T., Ban, F., Norinder, U., Gleave, M. E., and Cherkasov, A. Deep docking: a deep learning platform for augmentation of structure based drug discovery. *ACS central science*, 6(6):939–949, 2020.
- Graff, D. E., Shakhnovich, E. I., and Coley, C. W. Accelerating high-throughput virtual screening through molecular pool-based active learning. *Chemical science*, 12(22):7866–7881, 2021.
- Graff, D. E., Aldeghi, M., Morrone, J. A., Jordan, K. E., Pyzer-Knapp, E. O., and Coley, C. W. Self-focusing virtual screening with active design space pruning. *Journal of Chemical Information and Modeling*, 62(16):3854–3862, 2022.
- Griliches, Z. Errors in variables and other unobservables. *Econometrica: Journal of the Econometric Society*, pp. 971–998, 1974.
- Hughes, J. D., Blagg, J., Price, D. A., Bailey, S., De-Crescenzo, G. A., Devraj, R. V., Ellsworth, E., Fobian, Y. M., Gibbs, M. E., Gilles, R. W., et al. Physiochemical drug properties associated with in vivo toxicological outcomes. *Bioorganic & Medicinal Chemistry Letters*, 18(17):4872–4875, 2008.
- Klarich, K., Goldman, B., Kramer, T., Riley, P., and Walters, W. P. Thompson sampling: An efficient method for searching ultra-large synthesis on demand databases. *Journal of Chemical Information and Modeling*, 2024.
- Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8):30–37, 2009.
- Kosugi, T. and Ohue, M. Quantitative estimate of protein-protein interaction inhibitor accessibility of compounds: QEPI. *Journal of Chemical Information and Modeling*, 61(2):718–730, 2021.
- Krzyzanowski, A., Pahl, A., Grigalunas, M., and Waldmann, H. Spacial score—a comprehensive topological indicator for small-molecule complexity. *Journal of Medicinal Chemistry*, 66(18):12739–12750, 2023. doi: 10.1021/acs.jmedchem.3c00689.
- Landrum, G. et al. Rdkit: Open-source cheminformatics, 2006.
- Lipinski, C. A., Lombardo, F., Dominy, B. W., and Feeney, P. J. Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1-3):3–25, 1997.
- Luo, S., Gao, W., Wu, Z., Peng, J., Coley, C. W., and Ma, J. Projecting molecules into synthesizable chemical spaces. *arXiv preprint arXiv:2406.04628*, 2024.
- Mehta, S., Laghuvarapu, S., Pathak, Y., Sethi, A., Alvala, M., and Priyakumar, U. D. Memes: Machine learning framework for enhanced molecular screening. *Chemical science*, 12(35):11710–11721, 2021.
- Morrison, A., Friedland, G., and Wallach, I. CUina: An efficient GPU implementation of AutoDock Vina. In *American Chemical Society Fall 2020 Virtual Meeting & Expo*, August 2020.
- Pedawi, A., Gniewek, P., Chang, C., Anderson, B., and van den Bedem, H. An efficient graph generative model for navigating ultra-large combinatorial synthesis libraries. *Advances in Neural Information Processing Systems*, 35:8731–8745, 2022.
- Pedawi, A., De Oliveira, S., and van den Bedem, H. Through the looking glass: Navigating in latent space to optimize over combinatorial synthesis libraries. In *NeurIPS 2023 Generative AI and Biology (GenBio) Workshop*, 2023. URL <https://openreview.net/forum?id=ALsSkaldB3>.
- Rendle, S. Factorization machines. In *2010 IEEE International Conference on Data Mining (ICDM)*, pp. 995–1000. IEEE, 2010. doi: 10.1109/ICDM.2010.127.
- Sadybekov, A. A., Sadybekov, A. V., Liu, Y., Iliopoulos-Tsoutsouvas, C., Huang, X.-P., Pickett, J., Houser, B., Patel, N., Tran, N. K., Tong, F., et al. Synthon-based

- ligand discovery in virtual libraries of over 11 billion compounds. *Nature*, 601(7893):452–459, 2022.
- Singh, R., Sledzieski, S., Bryson, B., Cowen, L., and Berger, B. Contrastive learning in protein language space predicts interactions between drugs and protein targets. *Proceedings of the National Academy of Sciences*, 120(24): e2220778120, 2023.
- Thompson, W. R. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3-4):285–294, 1933.
- Tingle, B. I., Tang, K. G., Castanon, M., Gutierrez, J. J., Khurelbaatar, M., Dandarchuluun, C., Moroz, Y. S., and Irwin, J. J. ZINC-22: A free multi-billion-scale database of tangible compounds for ligand discovery. *Journal of Chemical Information and Modeling*, 63(4):1166–1176, 2023.
- Trott, O. and Olson, A. J. AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2):455–461, 2010.
- Veber, D. F., Johnson, S. R., Cheng, H.-Y., Smith, B. R., Ward, K. W., and Kopple, K. D. Molecular properties that influence the oral bioavailability of drug candidates. *Journal of Medicinal Chemistry*, 45(12):2615–2623, 2002.
- Wager, T. T., Hou, X., Verhoest, P. R., and Villalobos, A. Moving beyond rules: the development of a central nervous system multiparameter optimization (cns mpo) approach to enable alignment of druglike properties. *ACS chemical neuroscience*, 1(6):435–449, 2010.
- Zhang, J., Naruse, A., Li, X., and Wang, Y. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701092. doi: 10.1145/3581784.3607062. URL <https://doi.org/10.1145/3581784.3607062>.

## A. Appendix

### A.1. Combinatorial Synthesis Libraries

A combinatorial synthesis library (CSL),  $\mathcal{D} \equiv (\mathcal{T}, \mathcal{R}, \mathcal{S}, \psi, \varphi, \phi)$ , is organized hierarchically across three levels: synthons  $\mathcal{S}$  at the bottom, R-groups  $\mathcal{R}$  in the middle, and reactions  $\mathcal{T}$  at the top.

**Synthons.** A synthon  $s \in \mathcal{S}$  is a molecular building block, i.e., a molecular fragment with annotated attachment points, represented as a molecular graph or SMILES string  $x_s \in \mathcal{X}^*$ , where  $\mathcal{X}^* \supset \mathcal{X}$  extends the space of complete molecules to include attachment point tokens.

**R-groups.** An R-group  $r \in \mathcal{R}$  is a placeholder component in a multi-component reaction, spanned by a set of eligible synthons. The function  $\psi_{\mathcal{R} \rightarrow \mathcal{S}} : \mathcal{R} \rightarrow \mathcal{P}(\mathcal{S})$  returns the set of synthons  $\psi_{\mathcal{R} \rightarrow \mathcal{S}}(r) \subset \mathcal{S}$  that are valid assignments for a given R-group  $r$ , where  $\mathcal{P}(\cdot)$  denotes the power set.

**Reactions.** A reaction  $t \in \mathcal{T}$  is a multi-component synthesis rule that combines one synthon per R-group to produce a single product molecule. The function  $\psi_{\mathcal{T} \rightarrow \mathcal{R}} : \mathcal{T} \rightarrow \mathcal{P}(\mathcal{R})$  returns the set of R-groups  $\psi_{\mathcal{T} \rightarrow \mathcal{R}}(t) \subset \mathcal{R}$  associated with reaction  $t$ .

Each product in  $\mathcal{D}$  is uniquely identified by a multi-index  $\chi = (t, \{(r, s) : s \in \psi_{\mathcal{R} \rightarrow \mathcal{S}}(r), \forall r \in \psi_{\mathcal{T} \rightarrow \mathcal{R}}(t)\})$  describing its reaction and R-group assignment, and the synthesis rule  $\phi$  maps this multi-index to a product molecule  $x := \phi(\chi)$ .

Due to the combinatorial design of CSLs, the number of products grows multiplicatively with the number of synthons per R-group. For example, a library containing a single three-component reaction with 10,000 synthons per R-group enumerates a chemical space of one trillion products from only 30,000 synthons ( $10,000^3 = 10^{12}$ ). Commercially available CSLs like Enamine REAL achieve tens of billions of synthetically accessible products from a few hundred thousand synthons distributed across hundreds of reactions.

**Encoder hierarchy.** The APEX factorizer mirrors this three-level hierarchy with a corresponding stack of neural network encoders (see Section 3.2 for full details). At the bottom, the `SynthonEncoder` processes the molecular graph of each synthon  $s$  to produce a synthon-level embedding:

$$h_s^{\mathcal{S}} = \text{SynthonEncoder}(x_s). \quad (21)$$

A deep set network, the `RgroupEncoder`, then aggregates the synthon embeddings belonging to each R-group  $r$ :

$$h_r^{\mathcal{R}} = \text{RgroupEncoder}(\{h_s^{\mathcal{S}} : \forall s \in \psi_{\mathcal{R} \rightarrow \mathcal{S}}(r)\}). \quad (22)$$

Finally, another deep set network, the `ReactionEncoder`, aggregates the R-group embeddings belonging to each reaction  $t$ :

$$h_t^{\mathcal{T}} = \text{ReactionEncoder}(\{h_r^{\mathcal{R}} : \forall r \in \psi_{\mathcal{T} \rightarrow \mathcal{R}}(t)\}). \quad (23)$$

These representations form the basis for the `SynthonValueEncoder` and `RgroupKeyEncoder` that produce the synthon associative contributions  $v_{i,r,s}$  used in the APEX factorization ((17)). Once computed, these contributions are cached for all synthons in the library, so that scoring any compound at search time requires only a small, fixed number of additions regardless of library size. This encoding procedure scales as  $\mathcal{O}(|\mathcal{S}| + |\mathcal{R}| + |\mathcal{T}|)$ , which is logarithmic relative to the  $\mathcal{O}(|\mathcal{D}|)$  cost of encoding each product individually, and is performed once per library.

We refer the reader to Pedawi et al. (2022) for additional background.

### A.2. The Virtual Library

Figure 4 displays twenty randomly selected molecules from the 10B compound CSL constructed as part of this study. In Figure 5, the distribution of molecular properties for the fully enumerated 12M compound CSL are shown for both two- and three-component reactions. We note that compounds originating from three-component reactions tend to be larger than those from two-component reactions.

### A.3. Description of Computational Endpoints

Table 2 provides a full list of the molecular property and docking score endpoints considered in this study.

Docking scores against the five protein targets (PARP1, MET, DRD2, F10, and ESR1) were computed using CUina (Morrison et al., 2020), an efficient GPU implementation of AutoDock Vina (Trott & Olson, 2010), with receptor structures

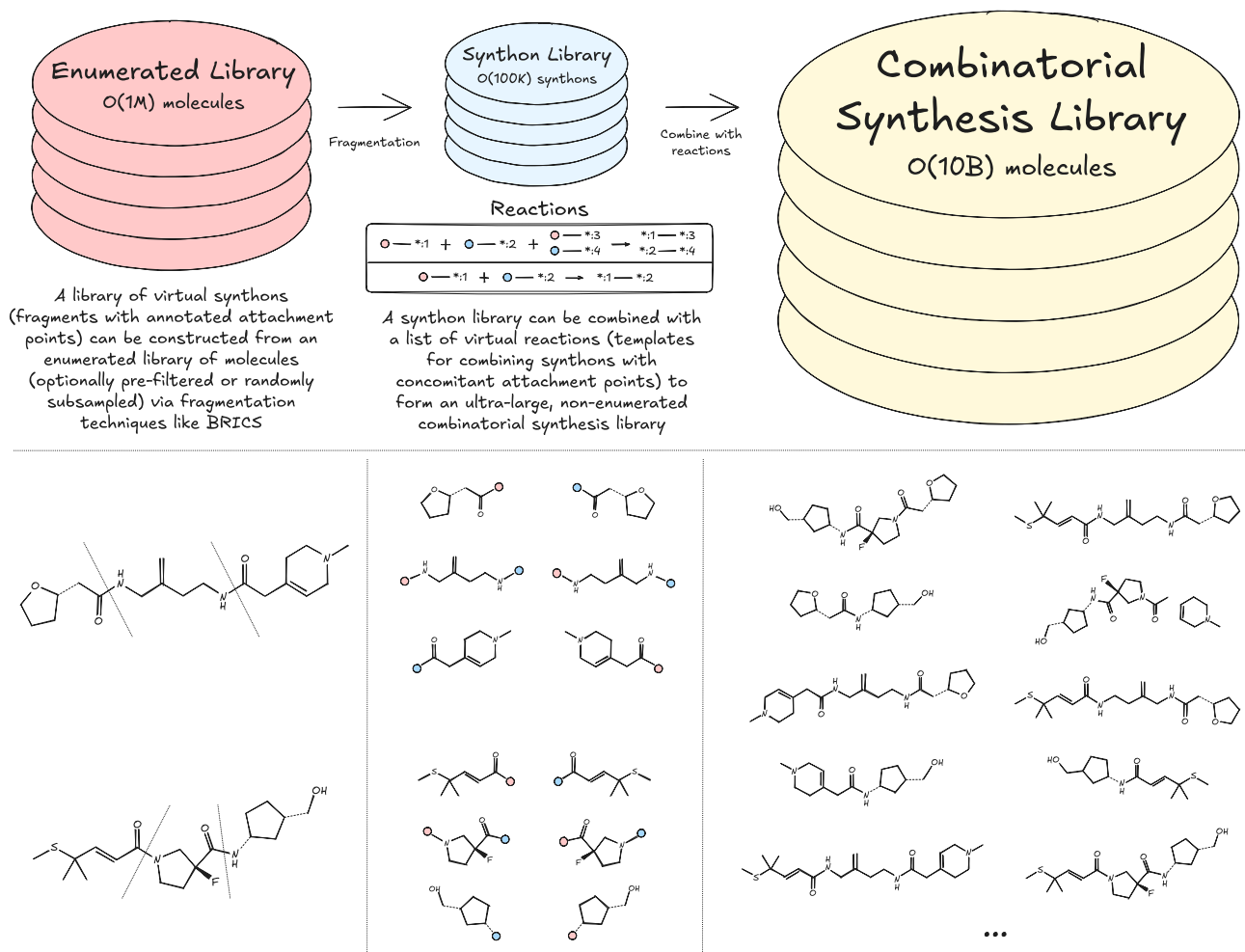


Figure 3. Beyond commercially available make-on-demand CSLs, it is relatively straightforward to design an ultra-large CSL for virtual screening using publicly available libraries of enumerated compounds like ZINC22 and cheminformatics tools like RDKit. These designs are incredibly valuable for virtual screening due to their ability to densely cover large swaths of relevant chemical space.

and binding site definitions taken from the DOCKSTRING dataset (García-Ortegón et al., 2022). The physicochemical properties were computed using RDKit (Landrum et al., 2006) or RDKit-based calculators.

All endpoints were computed on the fully enumerated 1M and 12M compound BRICS-based CSLs and are used throughout this study both as optimization objectives and as constraints in virtual screening queries.

#### A.4. Regression Performance by Endpoint

Figure 6 displays the R-squared for the surrogate (original and APEX-factorized) across all 28 endpoints considered in this study measured on a random sample of compounds from the 12M fully enumerated CSL. For the five docking score endpoints we additionally report rank correlation with Kendall’s tau-b and Spearman’s rho.

#### A.5. GPU Implementation of Factorized Top- $k$ Search

The factorized top- $k$  search employed in APEX is particularly well suited for GPUs. Each operation (score calculation, element tracing, and index decoding) can be performed independently for each compound in the CSL. Moreover, NVIDIA’s CCCL library (CCCL Development Team, 2023) provides an efficient batch-based AIR top- $k$  method (Zhang et al., 2023), which we leverage in our implementation using a chain-of-batches strategy.

We first partition the CSL into batches of (reaction, first R-group assignment) pairs of some chunk size (e.g., one billion

Table 2. Description of all molecular properties and docking score endpoints used in this study.

| Variable name                     | Description                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------------------|
| <i>Docking scores</i>             |                                                                                                    |
| score_PARP1                       | Docking score against PARP1 (poly ADP-ribose polymerase 1, an enzyme)                              |
| score_DRD2                        | Docking score against DRD2 (dopamine receptor D2, a GPCR)                                          |
| score_MET                         | Docking score against MET (hepatocyte growth factor receptor, a kinase)                            |
| score_F10                         | Docking score against F10 (coagulation factor Xa, a protease)                                      |
| score_ESR1                        | Docking score against ESR1 (estrogen receptor alpha, a nuclear receptor)                           |
| <i>physicochemical properties</i> |                                                                                                    |
| mol_wt                            | Molecular weight (Da)                                                                              |
| logp                              | Wildman-Crippen partition coefficient (lipophilicity)                                              |
| tpsa                              | Topological polar surface area ( $\text{\AA}^2$ )                                                  |
| n_hbd                             | Number of hydrogen bond donors                                                                     |
| n_hba                             | Number of hydrogen bond acceptors                                                                  |
| n_rotatable_bonds                 | Number of rotatable bonds                                                                          |
| n_atoms                           | Total number of atoms (including hydrogens)                                                        |
| n_heavy_atom                      | Number of heavy (non-hydrogen) atoms                                                               |
| mr                                | Wildman-Crippen molar refractivity                                                                 |
| n_ring                            | Total number of rings                                                                              |
| n_aromatic_ring                   | Number of aromatic rings                                                                           |
| n_aliphatic_rings                 | Number of aliphatic rings                                                                          |
| n_heterocycles                    | Number of heterocyclic rings                                                                       |
| n_bridgehead_atoms                | Number of bridgehead atoms                                                                         |
| n_stereocenter                    | Number of specified stereocenters                                                                  |
| n_unspecified_stereocenter        | Number of unspecified stereocenters                                                                |
| fraction_cs_p3                    | Fraction of $sp^3$ carbon atoms (Fsp3)                                                             |
| max_ring_size                     | Size of the largest ring in the molecule                                                           |
| qed                               | Quantitative estimate of drug-likeness (Bickerton et al., 2012)                                    |
| geppi                             | Quantitative estimate of protein-protein interaction inhibitor drug-likeness (Kosugi & Ohue, 2021) |
| sps                               | Spacial score, a topological indicator of small-molecule 3D complexity (Krzyzanowski et al., 2023) |
| synthetic_accessibility_score     | Synthetic accessibility score (SA score) (Ertl & Schuffenhauer, 2009)                              |
| weird_fragment_score              | Score penalizing the presence of unusual or undesirable chemical fragments                         |

compounds) and evaluate scores, the two-dimensional pair  $(\hat{c}(\mathbf{x}), \hat{f}_0(\mathbf{x}))$  denoting the APEX-predicted constraint violation and objective value, for all compounds in a batch on the GPU. For example, a batch can contain all compounds from the first three reactions (all R-groups fully enumerated) and all compounds from the fourth reaction where the first R-group assigned one of the first five eligible synthons, such that the total number of products is less than or equal to the specified chunk size.

Within a batch, compound scores are computed in parallel: CUDA blocks iterate over (reaction, first R-group assignment) pairs, while threads loop through subsequent R-group assignments. Synthon associative contributions are accumulated in shared memory for higher compute throughput. If a reaction has more than two R-groups, the remaining ones are processed with plain loops.

After score computation, results are passed to CCCL’s AIR top- $k$  method to filter for the top- $k$  indices for that batch. For subsequent batches, previously selected elements are prepended to the score arrays before the next AIR top- $k$  call, and the indices within the full CSL are tracked, enabling chain-of-batches. We carefully trace the movement of elements: an index larger than  $k$  means a new element from the current batch is within top  $k$ ; otherwise the element is from previous batches but its location within top  $k$  could have shifted. The kept indices array is updated accordingly.

Once the CSL is exhausted, each of the global top- $k$  indices within the library is decoded, again on GPU, using the (reaction, R-group assignment) mapping. The final results are then returned to the user for downstream processing (e.g., conversion of reaction and R-group assignment to SMILES).

#### A.6. Error Decomposition of the Factorized Surrogate

For a given endpoint, the total prediction error of the APEX factorized surrogate,  $\hat{f}_i$ , with respect to the ground truth oracle,  $f_i$ , can be decomposed into contributions from two distinct sources: the error of the surrogate  $\hat{f}_i$  with respect to the oracle, and the error introduced by the factorizer in approximating the surrogate. The total error decomposes additively as:

$$\hat{f}_i(x) - f_i(x) = \underbrace{(\hat{f}_i(x) - f_i(x))}_{\text{surrogate error}} + \underbrace{(\hat{f}_i(x) - \hat{f}_i(x))}_{\text{factorizer error}}. \quad (24)$$

Squaring and taking expectations yields the following decomposition of the expected squared error:

$$\mathbb{E} [(\hat{f}_i(x) - f_i(x))^2] = \mathbb{E} [(\hat{f}_i(x) - f_i(x))^2] + \mathbb{E} [(\hat{f}_i(x) - \hat{f}_i(x))^2] + 2 \mathbb{E} [(\hat{f}_i(x) - f_i(x))(\hat{f}_i(x) - \hat{f}_i(x))], \quad (25)$$

| Rule                                              | Property         | Value                                          |
|---------------------------------------------------|------------------|------------------------------------------------|
| <b>Lipinski Rule of 5</b> (Lipinski et al., 1997) | Molecular weight | $\leq 500$ Da                                  |
|                                                   | logP             | $\leq 5$                                       |
|                                                   | H-bond donors    | $\leq 5$                                       |
|                                                   | H-bond acceptors | $\leq 10$                                      |
| <b>Veber</b> (Veber et al., 2002)                 | Rotatable bonds  | $\leq 10$                                      |
|                                                   | TPSA             | $\leq 140 \text{ \AA}^2$                       |
| <b>Pfizer 3/75</b> (Hughes et al., 2008)          | logP             | $\leq 3$                                       |
|                                                   | TPSA             | $\geq 75 \text{ \AA}^2$                        |
| <b>Wager CNS</b> (Wager et al., 2010)             | Molecular weight | $\leq 360$ Da                                  |
|                                                   | logP             | $\leq 3$                                       |
|                                                   | TPSA             | $\geq 40 \text{ \AA}^2, \leq 90 \text{ \AA}^2$ |
|                                                   | H-bond donors    | $\leq 1$                                       |
| <b>Astex Rule of 3</b> (Congreve et al., 2003)    | Molecular weight | $\leq 300$ Da                                  |
|                                                   | logP             | $\leq 3$                                       |
|                                                   | H-bond donors    | $\leq 3$                                       |
|                                                   | H-bond acceptors | $\leq 3$                                       |
|                                                   | Rotatable bonds  | $\leq 3$                                       |
|                                                   | TPSA             | $\leq 60 \text{ \AA}^2$                        |

Table 3. Constraint sets used for experiments in Figure 2.

where the first term is the surrogate MSE, the second term is the factorizer MSE, and the third term is a cross term capturing the correlation between the two error sources.

If the surrogate error and the factorizer error are approximately uncorrelated, i.e., if the cross term in (25) is small in expectation, then the total MSE decomposes approximately as the sum of the two individual MSEs.

This decomposition is practically useful because each term can be reduced independently: the surrogate MSE is minimized by improving the molecular encoder  $g_\theta$  or by training on more labeled data, while the factorizer MSE is minimized by improving the `ReactionFactorizer`  $\hat{g}_\lambda$  or by training it on a larger or more diverse set of (reaction, R-group assignment) pairs — crucially, without requiring any additional oracle labels.

Figure 8 plots the decomposition of the normalized MSE across all 28 endpoints. We note that the cross term is negligible across all endpoints, confirming that the surrogate and factorizer errors are largely independent; this means that improvements to the surrogate encoder and the factorizer can be pursued separately to reduce the total NMSE. The relative contributions of the two error sources vary across endpoints: for some, such as the docking scores, the surrogate error (orange) dominates, while for others, the factorizer error (blue) is the primary contributor. This can serve as a guide for where architectural improvements are likely to be most impactful.

### A.7. Constraint Sets

Table 3 provides details on the constraints used in this paper’s experiments. We note that all of the computational endpoints and properties described in Table 2 can be used as objectives or constraints in an APEX query for the model trained as part of this paper.

### A.8. Architecture and Training Details

Below we provide a description of the model architectures and key hyperparameters used in this paper; the overall setup is intentionally lightweight and we did not perform extensive hyperparameter tuning, instead opting for sensible defaults. We refer the reader to the code repository (<https://github.com/NumerionLabs/apex>) for full implementation details.

**Message-passing neural network (MPNN).** Both the surrogate encoder and the factorizer’s synthon encoder are built on the same underlying MPNN architecture (implemented in `apex/nn/mpnn.py`), which operates on 2D molecular graphs. Each atom is featurized using a standard set of atomic features (atom type, degree, formal charge, chirality, aromaticity,

etc.) and each bond is featurized using bond-level features (bond type, ring membership, stereochemistry, etc.). These are projected into a  $d_{\text{node}}$ -dimensional node embedding and  $d_{\text{edge}}$ -dimensional edge embedding by learned linear layers. The network then applies  $L$  rounds of edge-conditioned message passing, in which each node aggregates messages from its neighbors, where each message is conditioned on the corresponding edge embedding. A graph-level representation is obtained by summing the final node embeddings and applying a final linear transformation to produce a  $d_{\text{graph}}$ -dimensional output. In all models used in this paper, all hidden dimensions are set to  $d = 64$  and the number of message-passing layers is  $L = 4$ .

**Surrogate.** The surrogate model  $g_\theta$  is a `LigandEncoder` (implemented in `apex/nn/encoder.py`) consisting of the MPNN described above, followed by a linear projection to the  $d = 64$ -dimensional embedding space. Task-specific predictions are obtained via a shared `LinearProbe` consisting of 28 task-specific linear heads (one per endpoint). The total parameter count for the surrogate encoder is 159,168. The probe adds  $28 \times 64 = 1,792$  weight parameters and 28 bias parameters, for an additional 1,820 parameters.

**Factorizer.** The `APEXFactorizer` (implemented in `apex/nn/apex.py`) mirrors the CSL hierarchy with a corresponding stack of encoders. At the bottom, a `SynthonGraphEncoder` — an instance of the same `LigandEncoder` MPNN architecture as the surrogate encoder — produces a  $d_{\text{hidden}} = 64$ -dimensional embedding for each synthon from its molecular graph. R-group embeddings are produced by a deep set network consisting of two `ResNetMLP` submodules with mean pooling applied between them over the synthons belonging to each R-group. Reaction embeddings are produced analogously by two further `ResNetMLP` submodules with sum pooling over the R-groups belonging to each reaction. Finally, a `RgroupAssociativeKeyEncoder` (a `ResNetMLP`) maps each (R-group, reaction) pair to a  $d \times d_U = 64 \times 64$  key matrix, and a `SynthonAssociativeValueEncoder` (a `ResNetMLP`) maps each synthon to a  $d_U = 64$ -dimensional value vector; their product gives the  $d = 64$ -dimensional synthon associative embedding  $u_{r,s}$ , as per (11). Each `ResNetMLP` uses  $d_{\text{hidden}} = 64$ ,  $n_{\text{layers}} = 4$ , and layer normalization. The total parameter count for the factorizer is 676,992.

**Training.** Training hyperparameters are specified in `configs/apex.yaml` in the associated code repository. Both models were trained using the Adam optimizer with learning rate  $10^{-3}$  for up to 100,000 iterations. The surrogate was trained on 80% of the 1M compound BRICS-based CSL (with the remaining 20% held out for validation). The factorizer was trained on the 12M compound BRICS-based CSL in the absence of labels, minimizing the reconstruction error of the frozen surrogate embeddings (13). For the 10B BRICS CSL, the factorizer was additionally fine-tuned on compounds from the library (using the same embedding reconstruction task).

### A.9. Comparison with Thompson Sampling

Figure 9 plots the recall of APEX against Thompson sampling across the five most prevalent reactions in the 12M compound CSL and against the five targets considered in this paper.

### A.10. Recall of Top- $j$ Compounds at Increasing Evaluation Budget

Figure 10 shows the recall of top compounds (in the absence of constraints) as a function of increasing evaluation budget, expressed as the fraction of the library evaluated with the oracle.

### A.11. Score-Based Constraints and Composite Objectives

To further test the robustness of the surrogate docking score predictions, we ran APEX search in a counter-screening scenario, where one target is chosen as the objective to minimize and constraints are added that the other four targets all score above their 50th percentile. Figure 11 shows the mean docking scores of the best 100 compounds (re-ranked by the true objective) after a  $k = 100,000$  search (BRICS 12M library), represented in terms of their eCDF. While these counter-screening constraints are generally effective at increasing the “selectivity” of the top compounds, they do result in worse absolute docking scores for the objective. We also tested defining a composite objective as the sum of all five targets’ docking scores, which proved quite effective at finding compounds that score well across all targets.



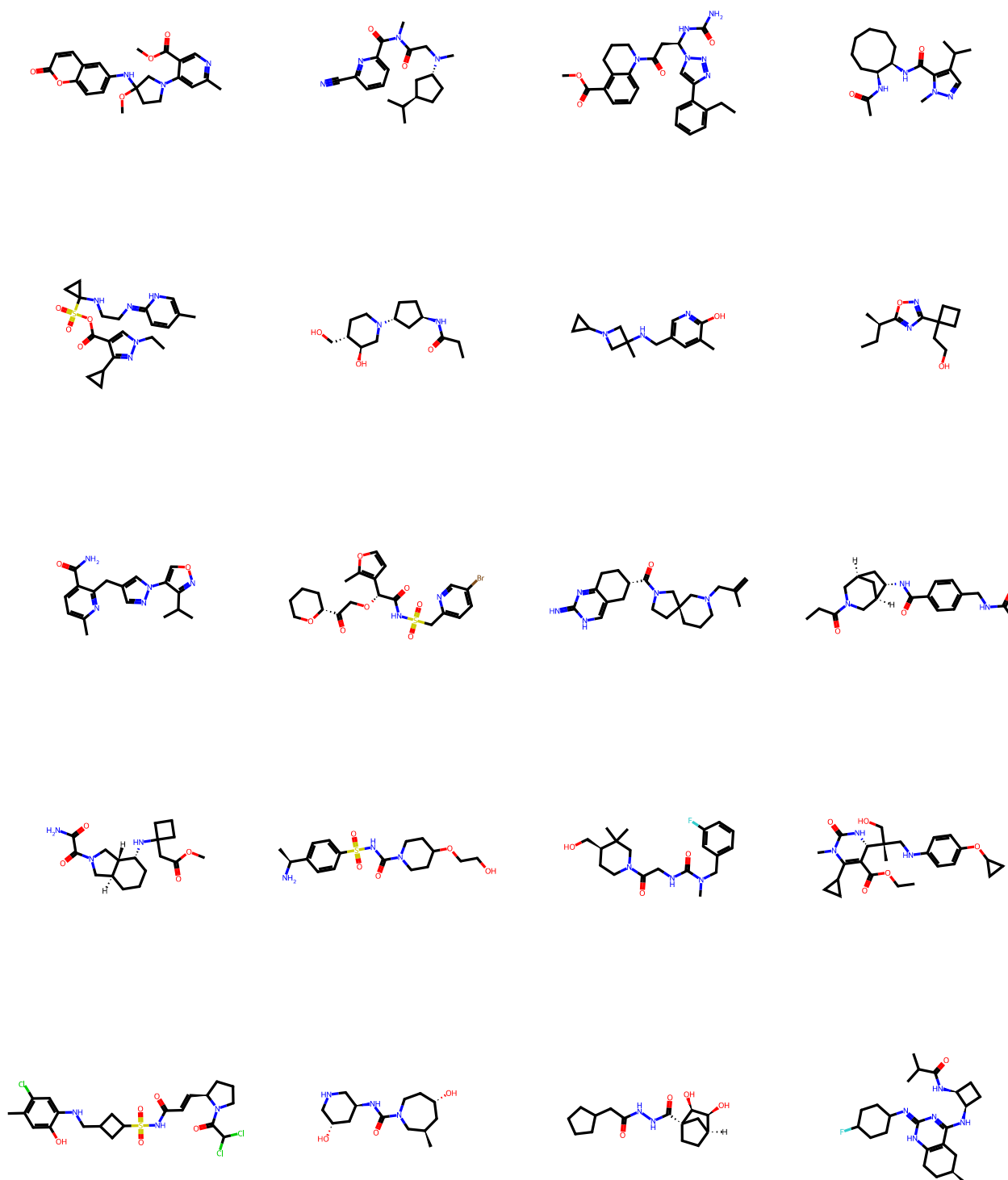


Figure 4. Example molecules from the 10B compound CSL.

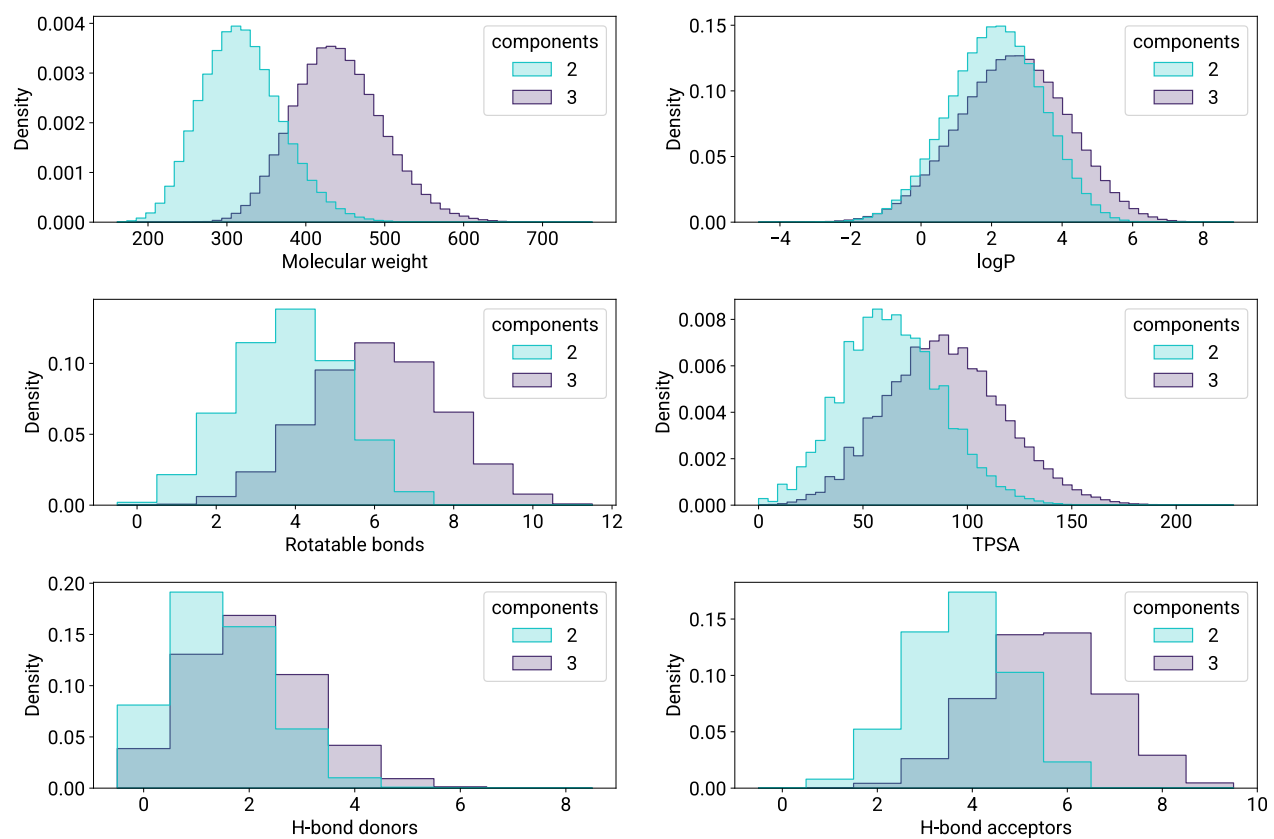


Figure 5. Distribution of select molecular properties in the 12M compound CSL.

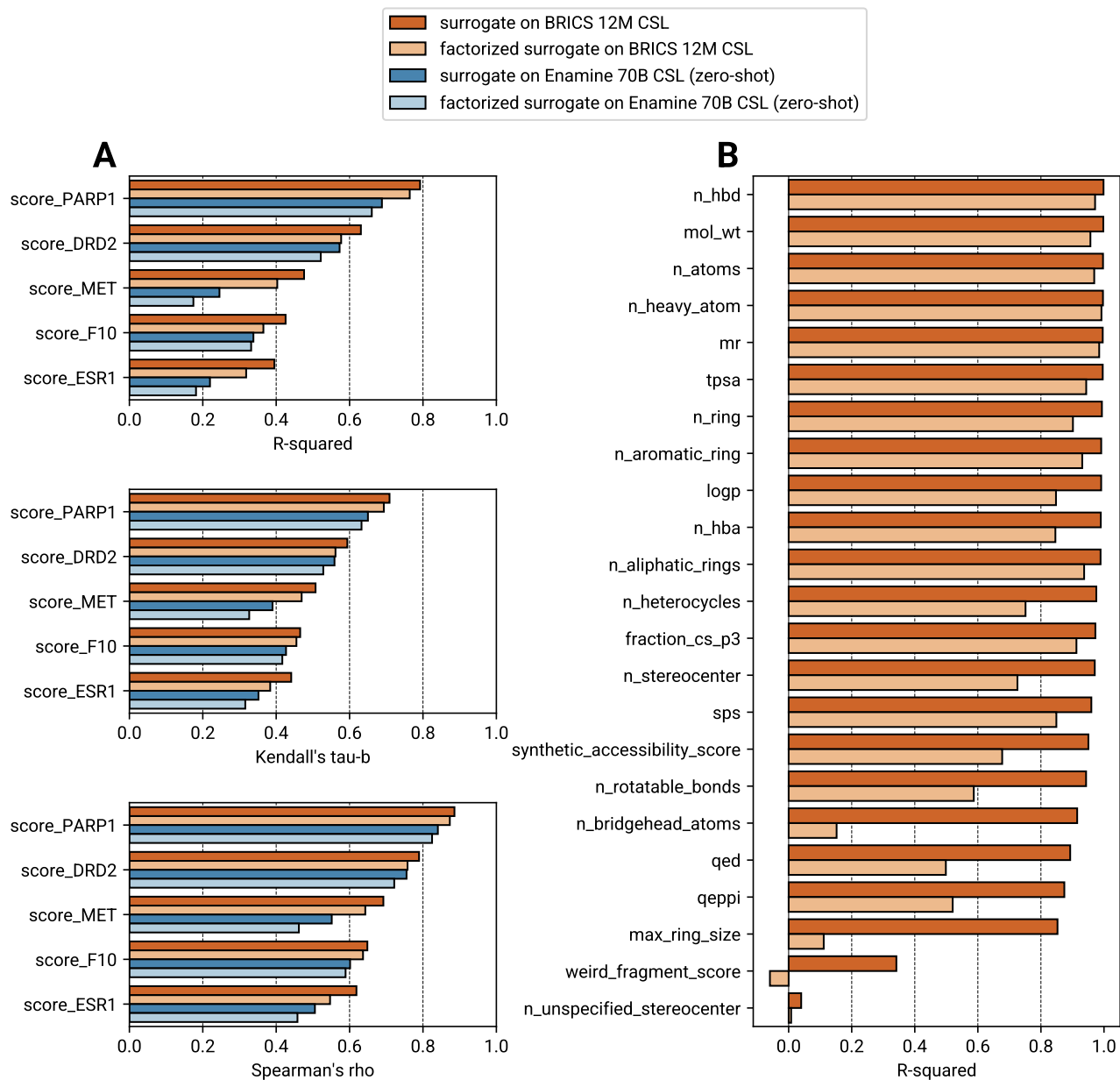


Figure 6. Accuracy of predicted (A) docking scores and (B) physicochemical properties for the original surrogate model as well as factorized version.

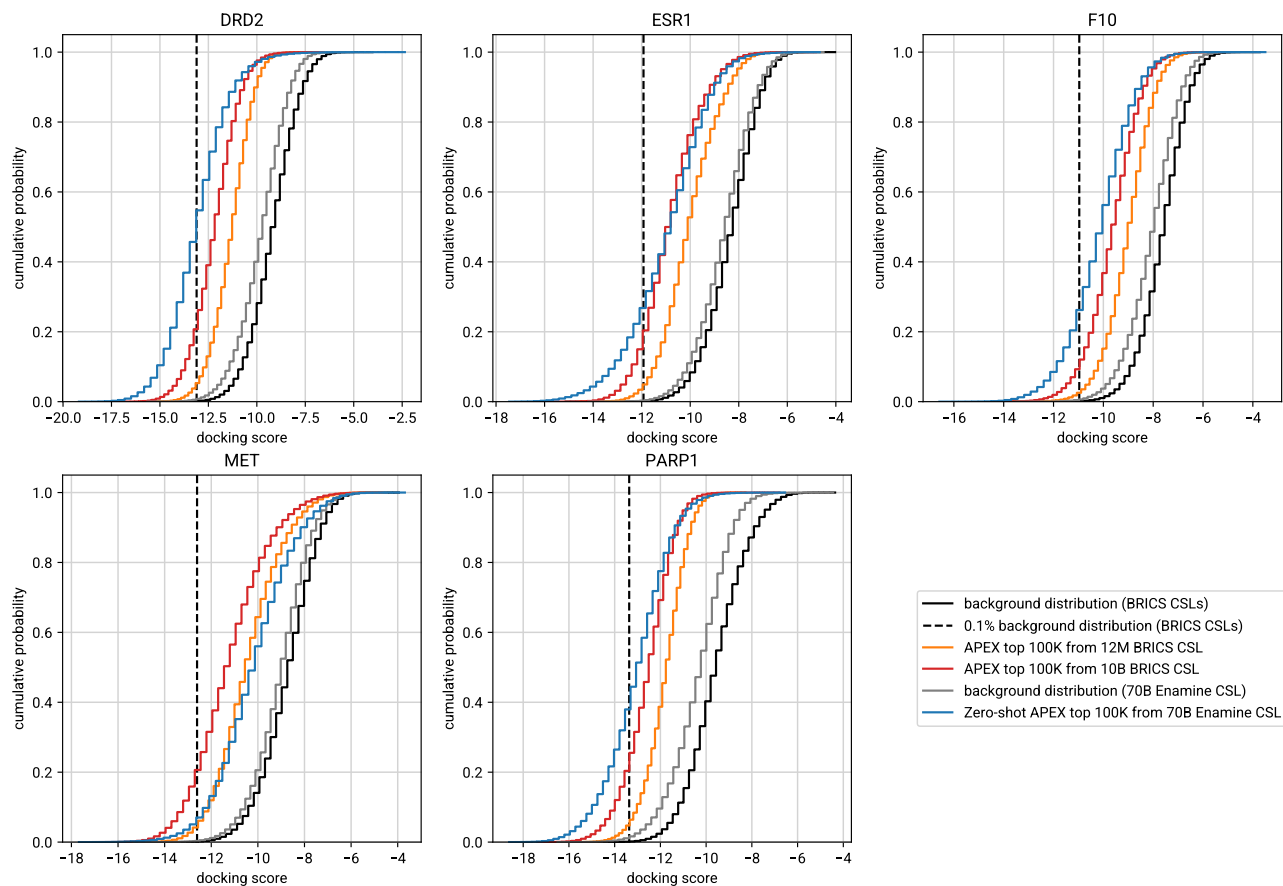


Figure 7. Docking scores for the APEX top- $k = 100,000$  on the 10B library are enriched with respect to the background distribution and with respect to the top- $k$  set from the smaller 12M library. Lower scores are better (i.e., indicate better interaction between ligand and receptor).

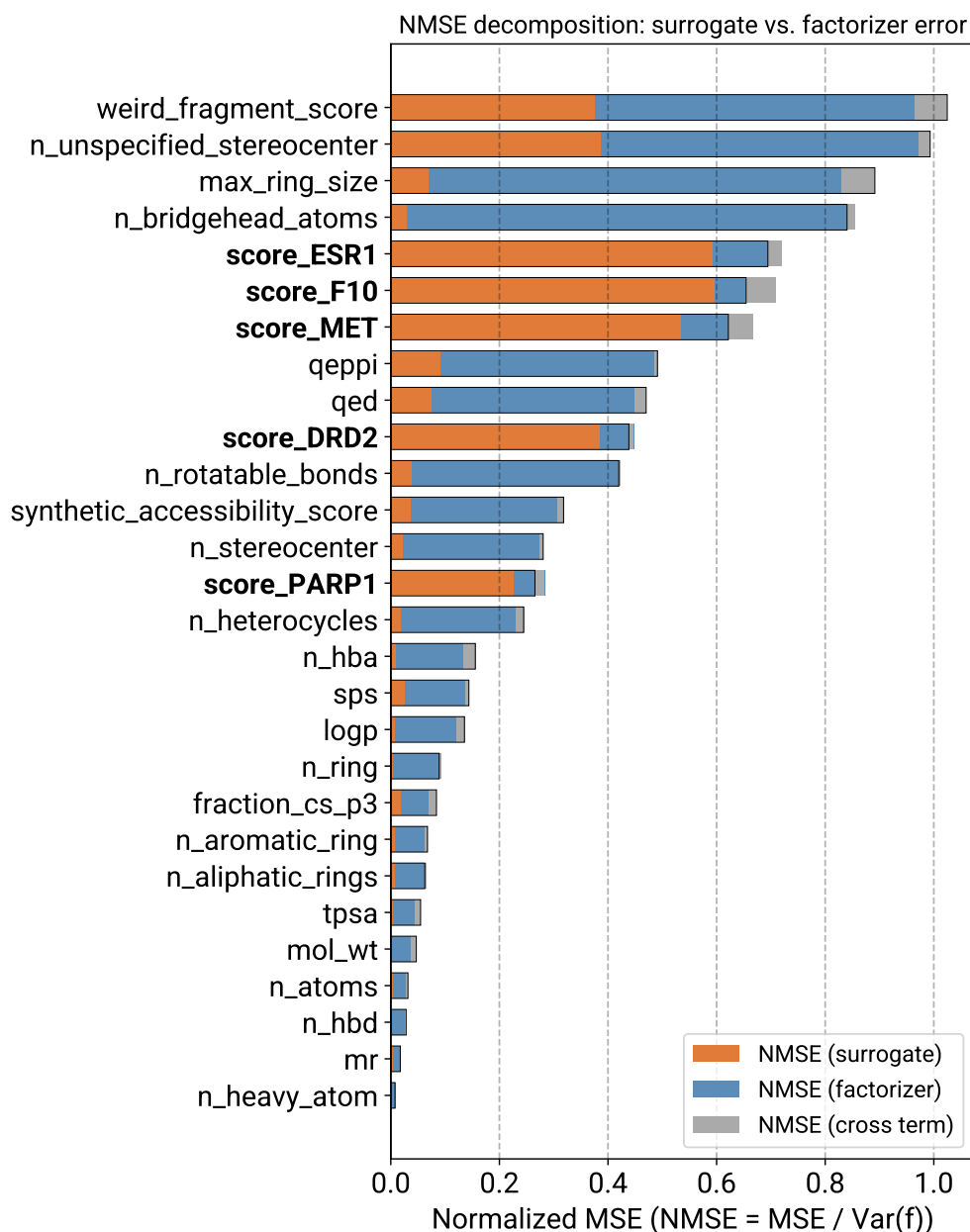


Figure 8. Normalized mean squared error (NMSE =  $\text{MSE} / \text{Var}(f)$ ) for the factorized surrogate  $\hat{f}_i$  with respect to the ground truth oracle  $f_i$ , decomposed into three additive components per endpoint as derived in Appendix A.6: surrogate error (orange), factorizer error (blue), and cross term (gray). The three components sum to the total NMSE for each endpoint, indicated by the black outline enclosing all three components. The cross term is negligible for most endpoints, indicating that the two error sources are largely independent and that the total NMSE is well approximated by the sum of the surrogate and factorizer MSEs alone.

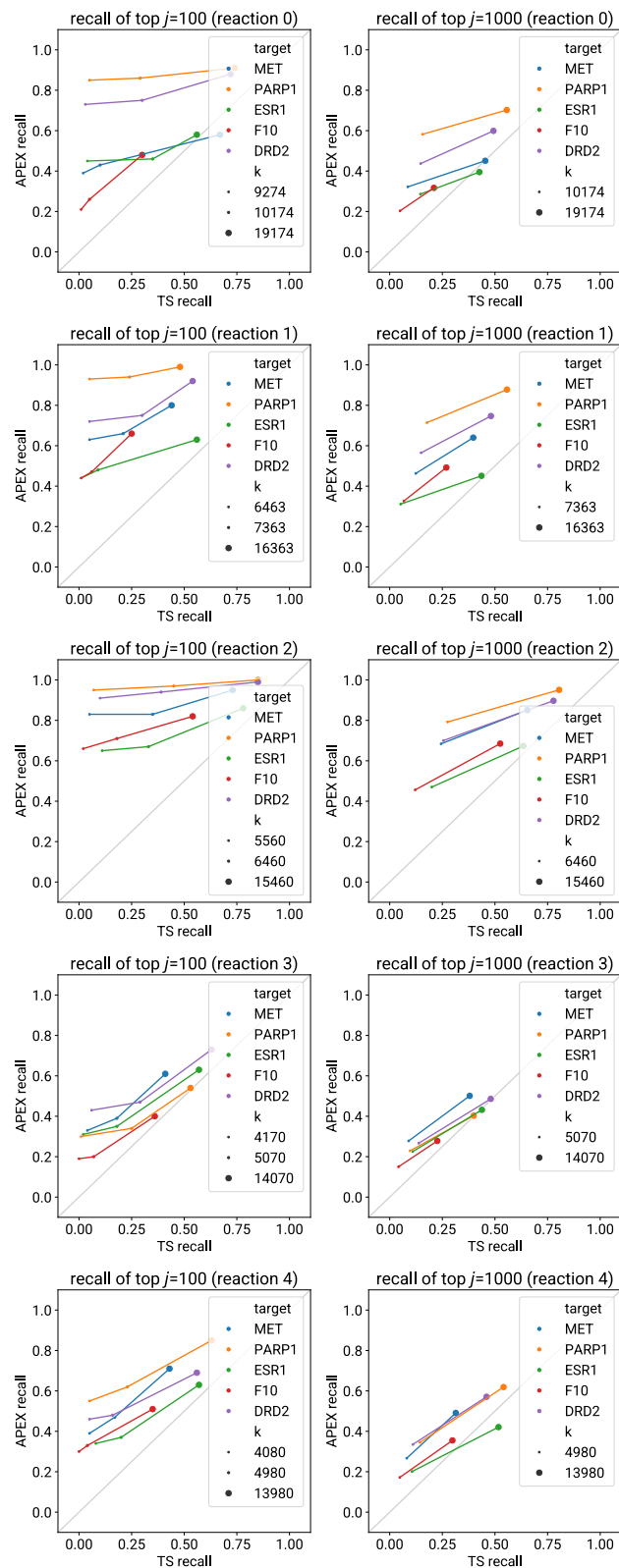


Figure 9. Top- $j$  recall for APEX and Thompson sampling (TS) using matched evaluation budgets. APEX search run using  $k$  set to the number of total evaluations for TS. Thompson sampling comparison was run using three and ten warmup steps for two- and three-component reactions, respectively, and 10, 1000, or 10,000 iterations of Thompson sampling.

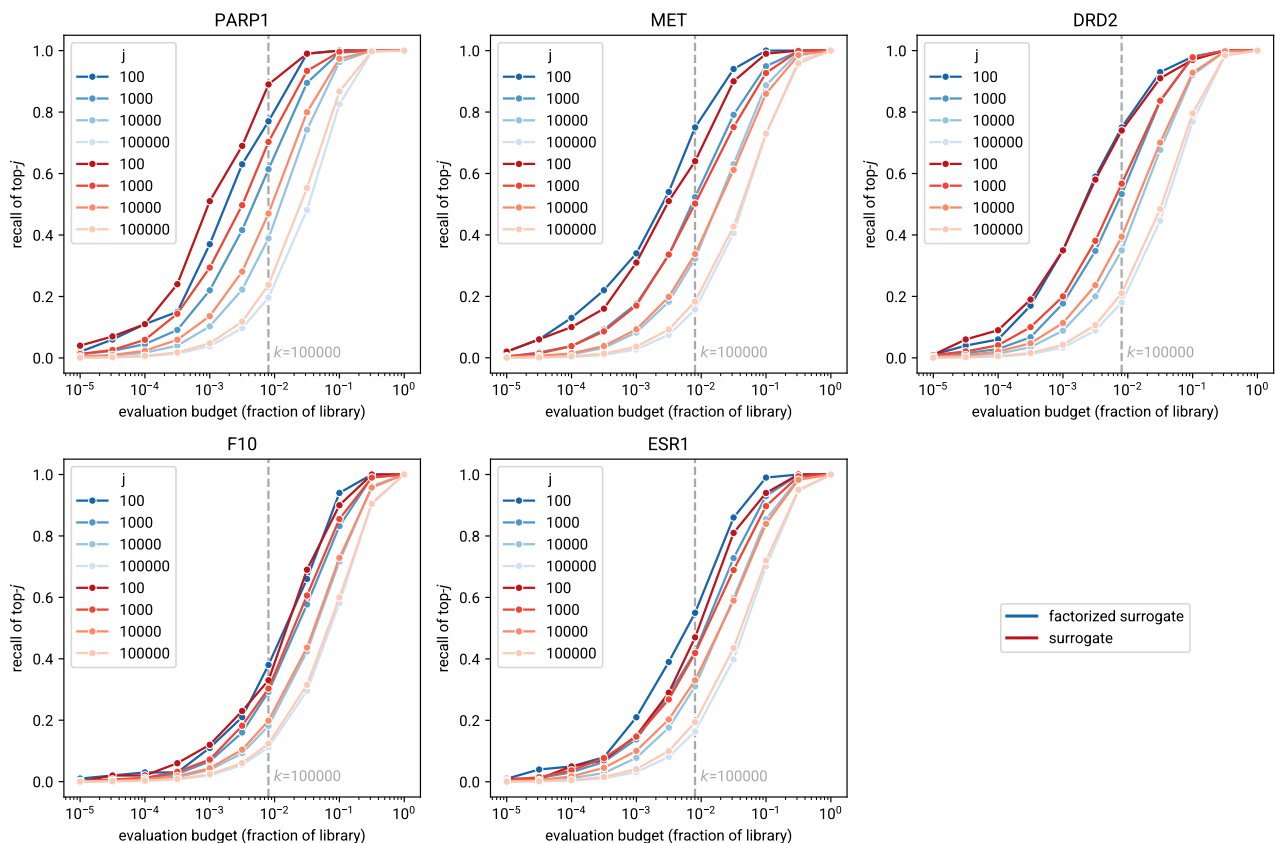


Figure 10. Recall of ground truth top- $j$  compounds by the surrogate model (red) and factorized surrogate (blue) at different evaluation budgets. No constraints were imposed. Dashed line corresponds to a budget of  $k = 100,000$  compounds, which was used for the evaluations in Figure 2.

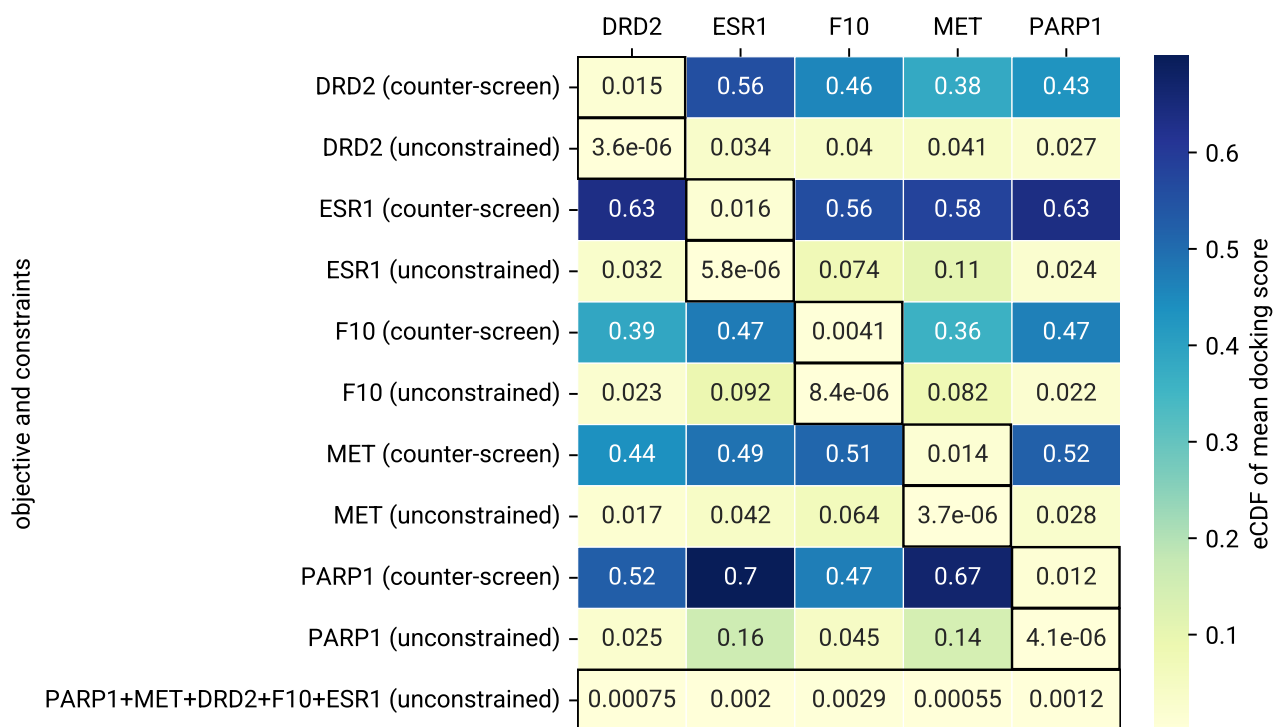


Figure 11. Inclusion of constraints on non-objective docking scores allows for APEX to be used in a counter-screening fashion. Each row is the result of a single APEX search ( $k = 100,000$ ), either unconstrained or with “counter-screening” constraints (non-objective docking scores  $> 50$ th percentile). Cells are outlined if they were used as the objective, and values are the eCDF of the mean docking score for the top 100 molecules after re-ranking by the true objective. (Last row) APEX search with composite objective of all five targets’ docking scores.